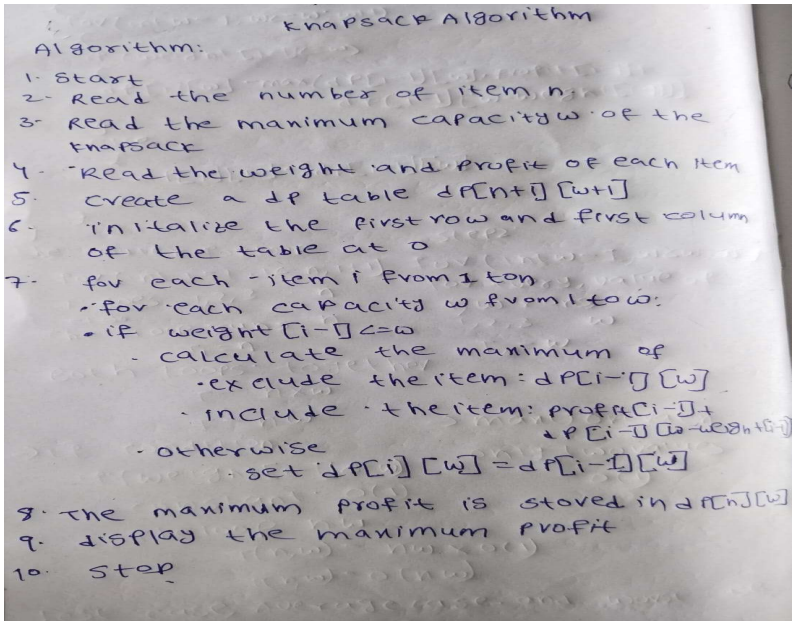
# Practical 5

Aim: Implementation of a knapsack problem using dynamic programming.

## Algorithm:




## Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

class Knapsack
{
private:
   int w[10];          // weights
```

```
    int v[10];         // values
    int n;             // number of items
    int W;             // capacity
    int dp[10][100];   // DP table

public:

    // Get Input
    void getInput()
    {
        cout << "Enter number of items: ";
        cin >> n;

        cout << "Enter knapsack capacity: ";
        cin >> W;

        for (int i = 1; i <= n; i++)
        {
            cout << "Item " << i
                 << " weight and value: ";
            cin >> w[i] >> v[i];
        }
    }

    // Build DP Table
    void solve()
    {
        // Base case: column 0
        for (int i = 0; i <= n; i++)
        {
            dp[i][0] = 0;
        }

        // Base case: row 0
        for (int j = 0; j <= W; j++)
        {
            dp[0][j] = 0;
        }
```

```
    // Fill DP table
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= W; j++)
        {
            if (w[i] <= j)
            {
                // Take or do not take the item
                dp[i][j] = max(
                    v[i] + dp[i - 1][j - w[i]],
                    dp[i - 1][j]
                );
            }
            else
            {
                // Item cannot be taken
                dp[i][j] = dp[i - 1][j];
            }
        }
    }
}

// Print DP Table
void printTable()
{
    cout << "\nDP Table:" << endl;

    cout << "     ";

    for (int j = 0; j <= W; j++)
    {
        cout << j << "  ";
    }

    cout << endl;

    for (int i = 0; i <= n; i++)
    {
        cout << "i=" << i << "  ";
```

```
        for (int j = 0; j <= W; j++)
        {
            cout << dp[i][j] << "  ";
        }

        cout << endl;
    }
}

// Find Selected Items
void findItems()
{
    cout << "\nSelected Items:" << endl;

    int j = W;

    for (int i = n; i >= 1; i--)
    {
        if (dp[i][j] != dp[i - 1][j])
        {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") TAKEN" << endl;

            j = j - w[i];
        }
        else
        {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") NOT taken" << endl;
        }
    }
}

// Show Result
```

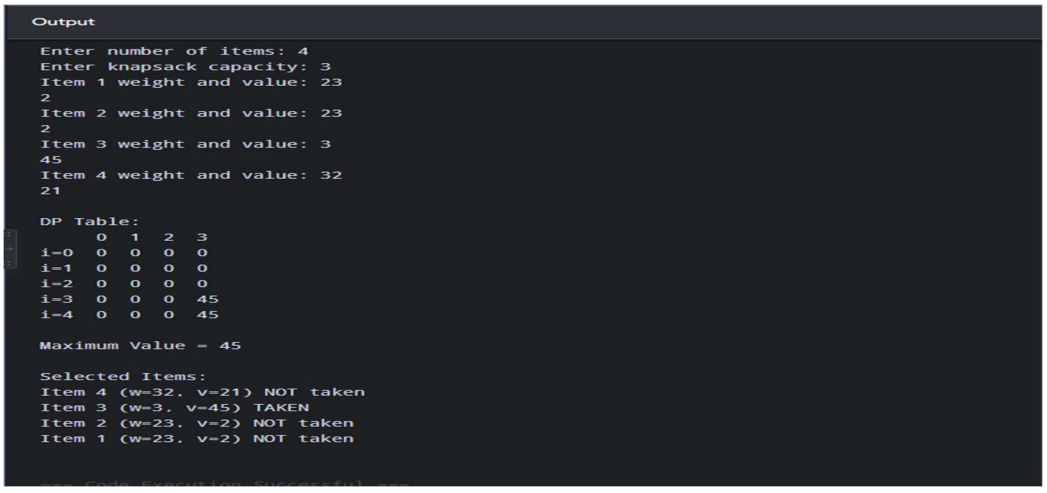
```
    void showResult()
    {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};

int main()
{
    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

## Output:



Time Complexities: